DIREZIONE PEOPLE & CULTURE · CAFFÈ BELMORO S.P.A.

Ottobre 2026

Chicco

Un agente AI ibrido e multi-tool per la funzione People & Culture

Un'unica chat per il manuale delle policy HR e per i dati dell'organico. È l'agente a decidere, domanda per domanda, dove andare a cercare la risposta.

A CURA DI

Paolo Savino

PERCORSO

Master in AI ed Agenti AI per il business

Caffè Belmoro S.p.A. è un'azienda inventata: scenario, dataset e policy sono fittizi, creati a scopo didattico.

LO SCENARIO

Due domande opposte, ogni giorno, nella stessa funzione

DOMANDA QUALITATIVA

«Quanti giorni di smart working spettano a chi lavora in IT?»

Dove sta la risposta

Nel manuale delle policy HR: 15 sezioni e 21 FAQ interne, fra smart working, ferie, welfare, retribuzione e performance.

Come funziona oggi

Si cerca nel documento, spesso si risponde a memoria. Le risposte non sono uniformi e nessuno può verificarle.

DOMANDA QUANTITATIVA

«Quale reparto ha il tasso di turnover più alto?»

Dove sta la risposta

Nell'anagrafica dell'organico: 1.240 dipendenti, 25 colonne fra retribuzione, performance, formazione, assenze e cessazioni.

Come funziona oggi

Serve un analista. I dati vanno prima ripuliti e il ciclo fra domanda e risposta si misura in giorni.

Nessuna delle due risposte è nelle mani di chi ha la domanda. E quando la domanda le riguarda entrambe, non esiste nemmeno un posto dove farla.

LA SOLUZIONE

Una sola chat. È l'agente a scegliere lo strumento.

Chi lavora in People & Culture scrive la domanda in italiano. L'agente capisce se si tratta di una regola o di una misura, va a prenderla alla fonte giusta e restituisce la risposta con la sua giustificazione.

4 – 25 s

Dalla domanda alla risposta

Una regola di policy torna in pochi secondi, un'analisi completa di grafico in meno di mezzo minuto. Al posto di una ricerca nel manuale o di una richiesta all'analista.

100%

Risposte verificabili

Ogni risposta mostra da dove viene: i passaggi del manuale citati con la loro sezione, oppure il codice Python effettivamente eseguito sui dati.

1.240

Dipendenti interrogabili

Turnover, retribuzioni, performance, formazione e smart working, già normalizzati: l'HR business partner non dipende più dall'analista per una domanda puntuale.

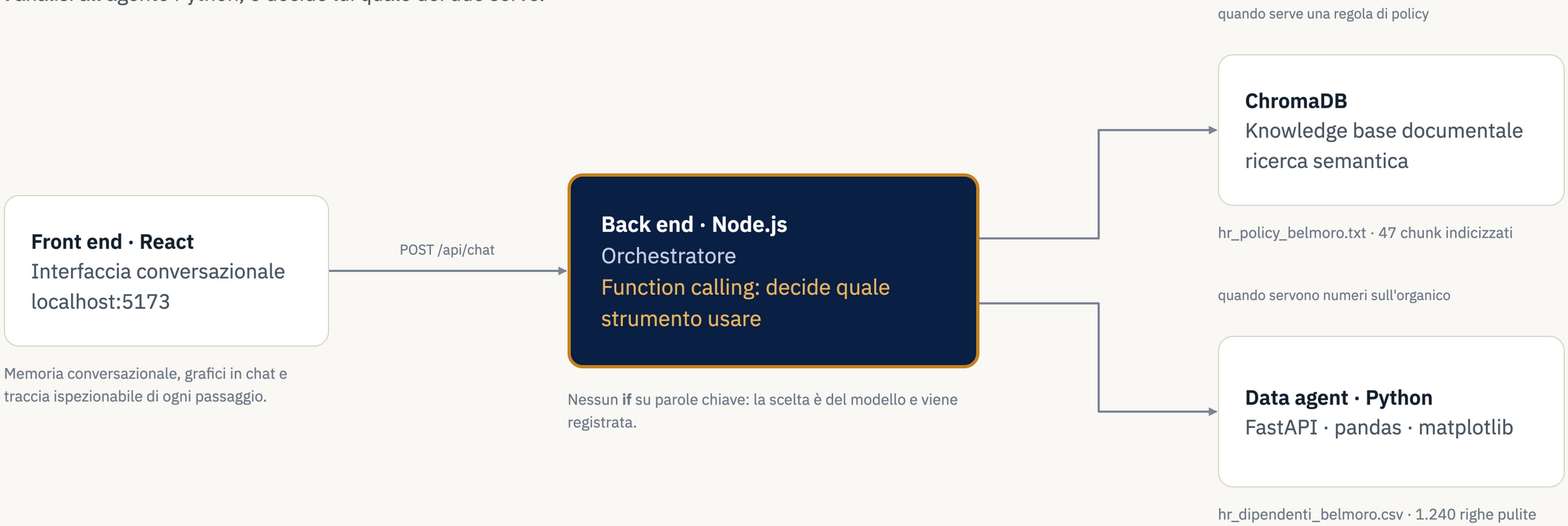
IL VERO SALTO

Quando la domanda tocca entrambi i mondi — «rispettiamo l'obiettivo sul gender pay gap?» — l'agente usa i due strumenti **in sequenza**: legge la soglia nella policy, misura il dato reale e mette a confronto regola e realtà.

L'ARCHITETTURA

Tre microservizi, un unico punto di ingresso

Il front end parla solo con il back end. Il back end non calcola nulla: interroga la knowledge base oppure delega l'analisi all'agente Python, e decide lui quale dei due serve.



Memoria conversazionale, grafici in chat e traccia ispezionabile di ogni passaggio.

Nessun **if** su parole chiave: la scelta è del modello e viene registrata.

I grafici prodotti dall'agente Python vengono salvati in una cartella condivisa e pubblicati dal back end su **/charts**: il front end li richiede a un solo indirizzo, senza sapere che esiste un servizio Python.

IL CUORE DEL SISTEMA

La decisione è del modello, non del codice

Il back end espone due funzioni e le descrive con precisione, poi lascia scegliere l'LLM in un ciclo **ReAct**: ragiona, agisce, osserva il risultato e itera. Sono possibili quattro esiti, tutti gestiti dallo stesso ciclo.

ESITO 1 · NESSUNO STRUMENTO

Risposta diretta

Saluti, domande sulle proprie capacità, richieste di chiarire qualcosa già detto. Nessuna chiamata, nessun costo inutile.

ESITO 2 · KNOWLEDGE BASE

Solo RAG sulle policy

Regole, procedure, diritti e soglie: smart working, ferie, welfare, revisione salariale, preavviso, trasferte.

ESITO 3 · AGENTE DATI

Solo analisi del dataset

Medie, distribuzioni, confronti fra reparti, trend temporali, tassi di turnover, correlazioni, anomalie nei dati.

ESITO 4 · I DUE IN SEQUENZA

Regola e dato a confronto

Legge prima la soglia nella policy, poi costruisce la richiesta di analisi **nei termini di quella soglia** — stessa unità, stesso livello di dettaglio — e confronta.

La qualità della scelta dipende quasi per intero da come sono descritte le due funzioni: cosa coprono (*«smart working, ferie, welfare, PIP…»*) e cosa non devono fare (*«non usarlo per calcoli o medie»*). Il resto lo fa il modello.

DEMO · DOMANDA SU UNA REGOLA

«Quanti giorni di smart working al mese posso fare se lavoro in IT?»

Knowledge base · RAG

La scelta

L'agente riconosce una domanda su una regola e interroga la knowledge base, senza toccare il dataset.

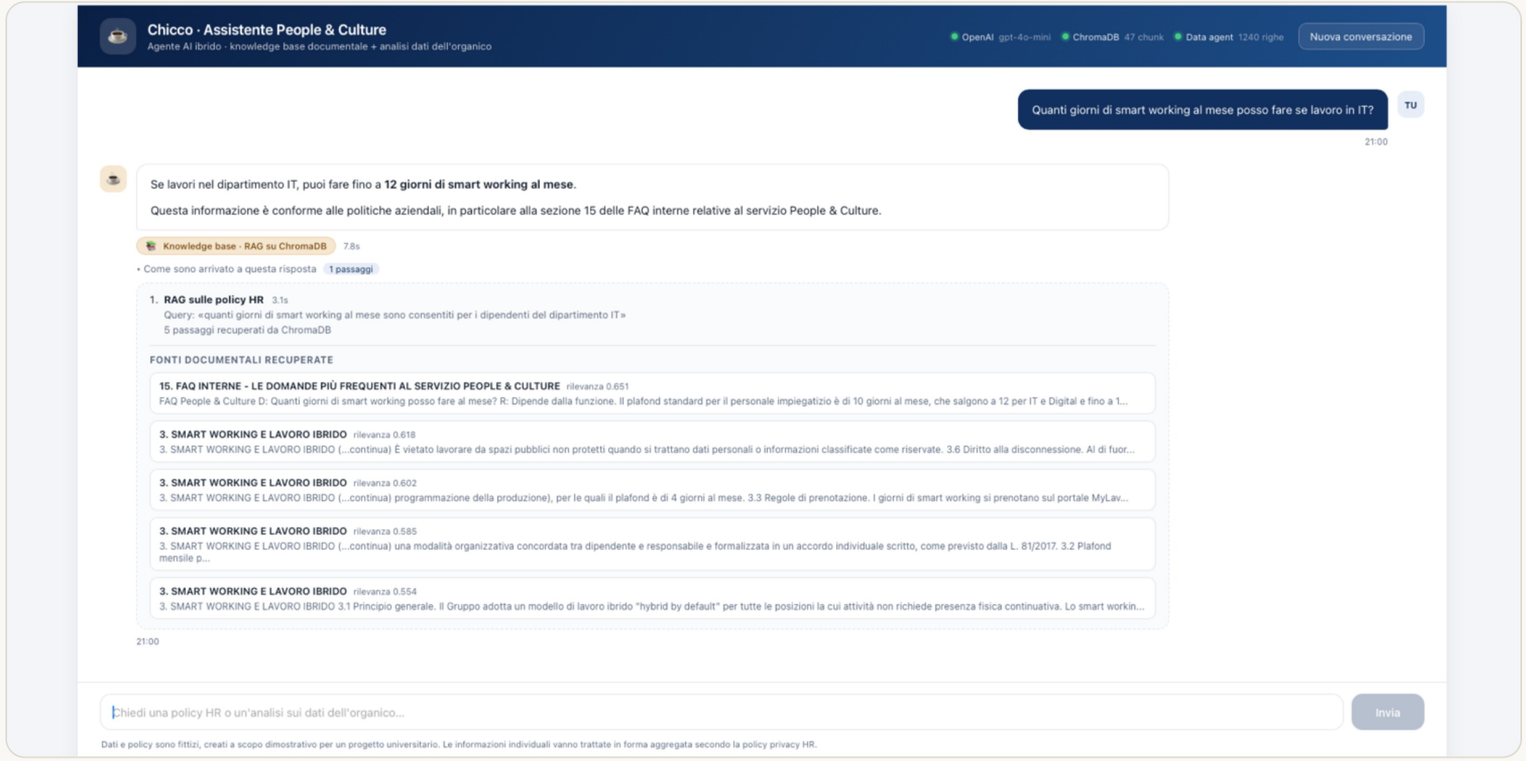
Il recupero

Cinque passaggi dal manuale, ordinati per rilevanza semantica. In cima la FAQ che risponde esattamente, con punteggio 0,651.

La risposta

12 giorni al mese, con il riferimento alla sezione del manuale. Il modello può usare solo i passaggi recuperati: la conoscenza generica gli è vietata.

Il pannello **«Come sono arrivato a questa risposta»** resta aperto a chi legge: la query, le fonti e il loro punteggio.



DEMO · DOMANDA SUI NUMERI

«Qual è la RAL media per dipartimento? Mostrami un grafico.»

Data agent · pandas

La delega

Il back end passa la richiesta al microservizio Python. L'agente pandas scrive il codice, lo esegue, osserva il risultato e itera.

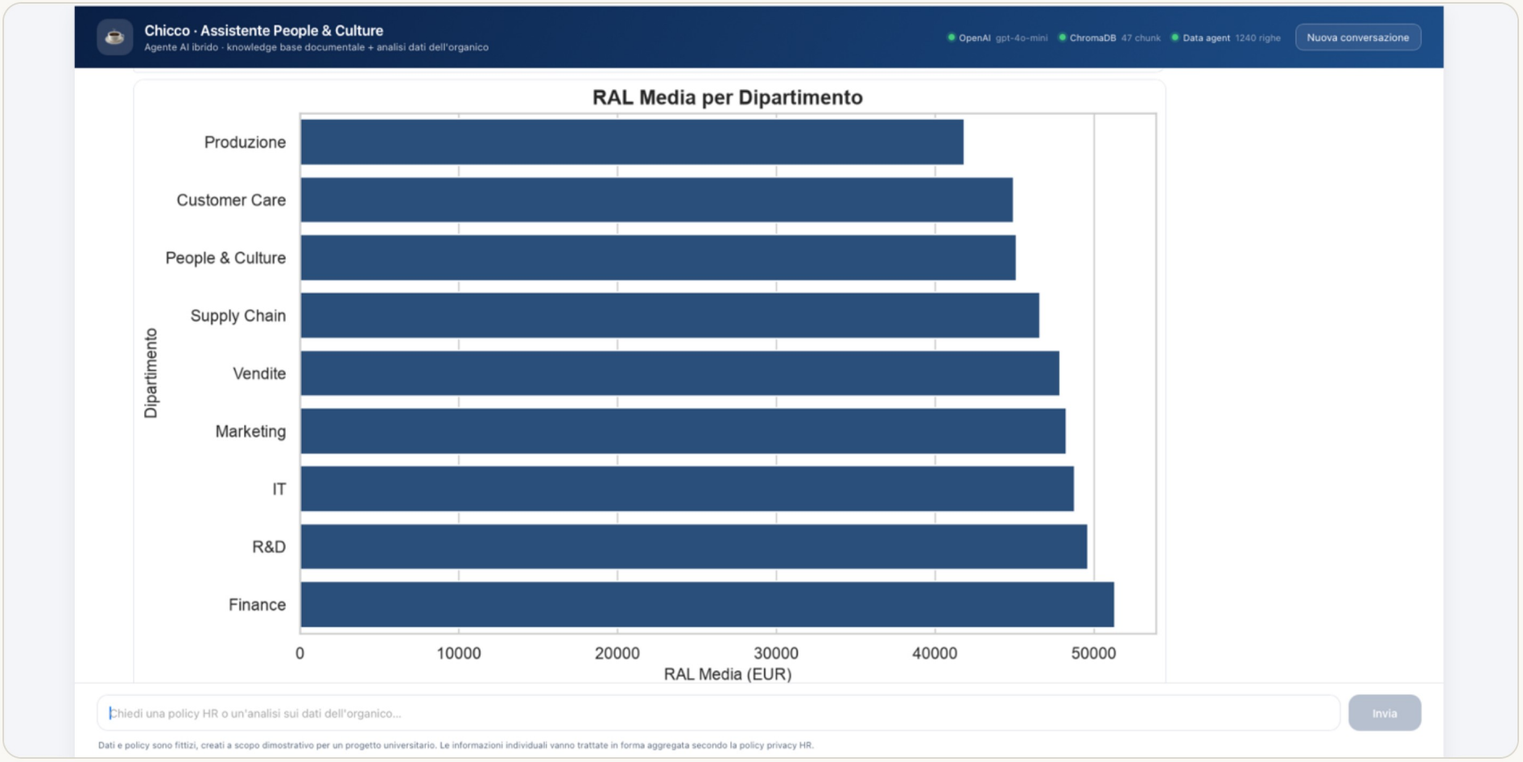
La pulizia

Prima del calcolo, 1.285 righe diventano 1.240: duplicati rimossi, date in quattro formati normalizzate, importi con valuta e separatori misti convertiti, valori fuori range esclusi.

Il risultato

Da 41.827 € in Produzione a 51.327 € in Finance, con il grafico generato da matplotlib e mostrato direttamente in chat.

La risposta non è solo una lista di numeri: si chiude sempre con una lettura manageriale e una nota su cosa è stato escluso dal calcolo.



DEMO · LA CONVERSAZIONE HA MEMORIA

«E per il Marketing?»

Data agent · pandas

Tre parole, nessun contesto esplicito

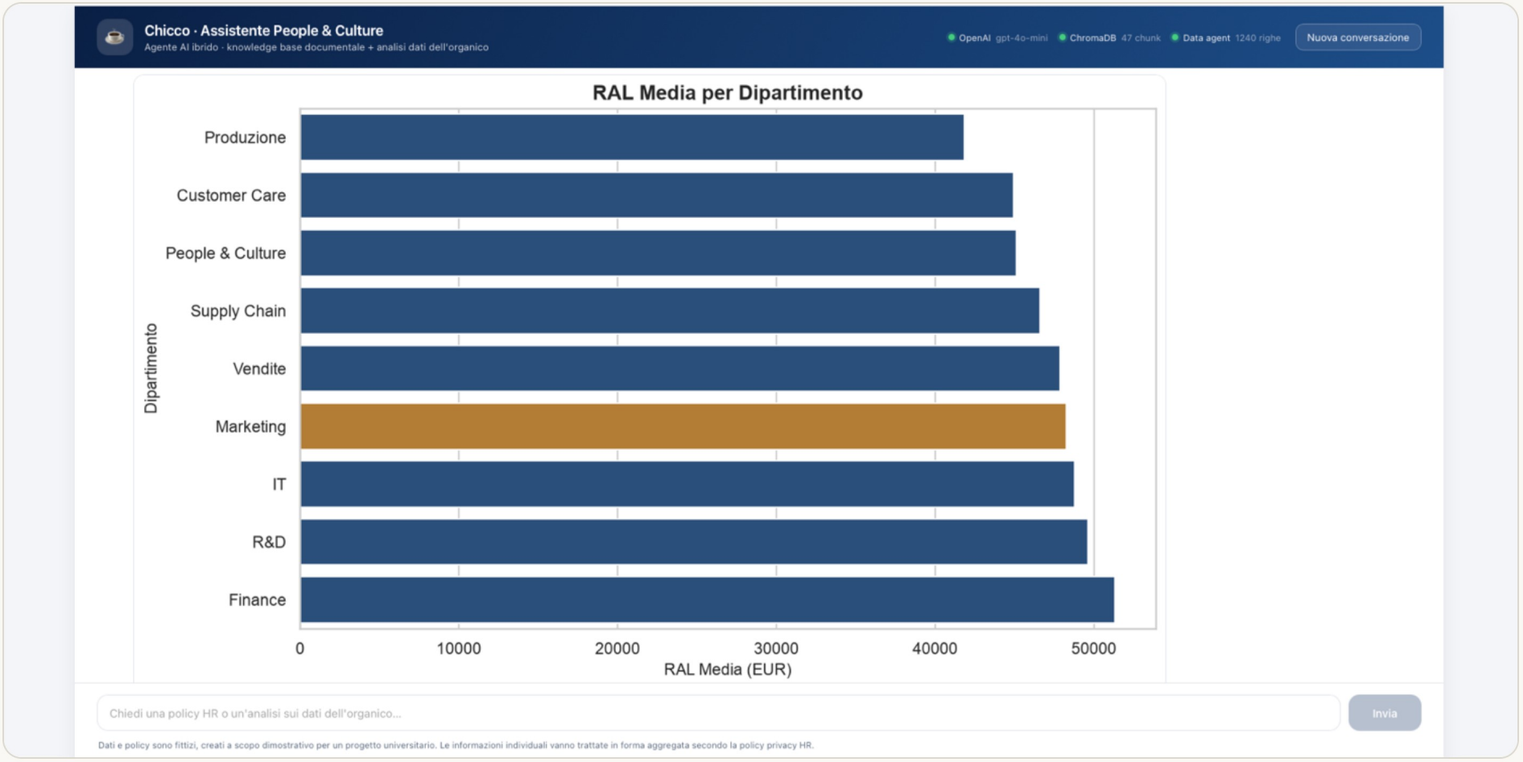
L'orchestratore risolve il riferimento implicito e riscrive la domanda in una richiesta autonoma per l'agente dati.

Richiesta inviata al data agent:
«calcola la RAL media per tutti i dipartimenti, evidenziando il Marketing nel grafico»

48.257,89 €, la sesta più alta

Il valore puntuale c'è, ma resta dentro il suo confronto: un numero isolato non dice a un responsabile se sia alto o basso.

Il dato viene sempre ricalcolato alla fonte, mai riletto dalle risposte precedenti.



DEMO · REGOLA E DATO NELLA STESSA RISPOSTA

«Il nostro gender pay gap rispetta l'obiettivo fissato dalla policy?»

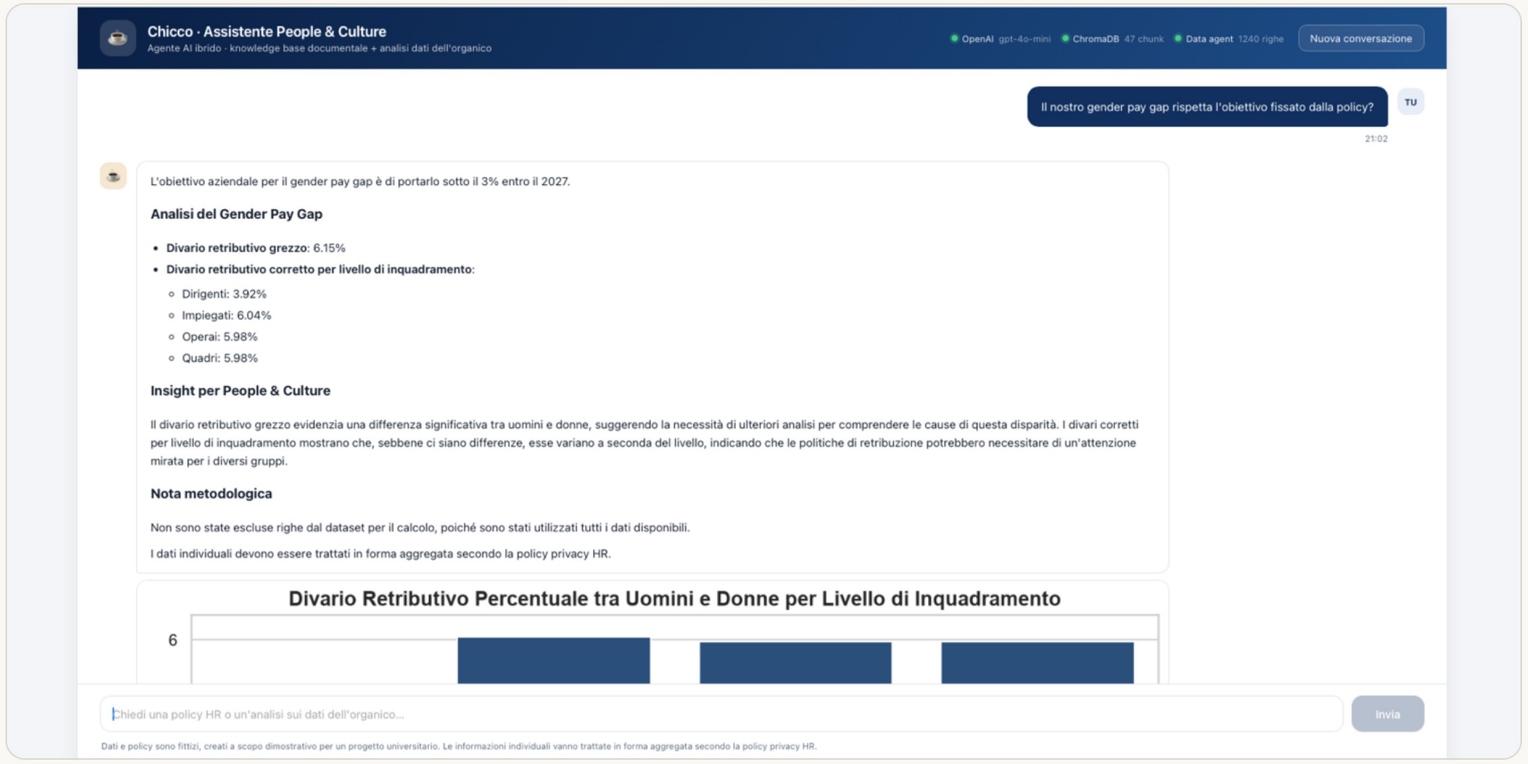
RAG + data agent

Dalla policy
Obiettivo sotto il **3% entro il 2027**, misurato sul divario corretto a parità di ruolo, livello e anzianità.

Dai dati
Divario grezzo **6,15%**. Per livello: 3,92% dirigenti, 6,04% impiegati, 5,98% operai e quadri.

Il verdetto
Obiettivo non ancora raggiunto a nessun livello di inquadramento, con il divario più ampio proprio sulla popolazione più numerosa, gli impiegati.

Una domanda di questo tipo, oggi, richiede una richiesta formale a People Analytics e qualche giorno di attesa.



DENTRO LA DECISIONE

Due strumenti, un solo turno, tutto tracciato

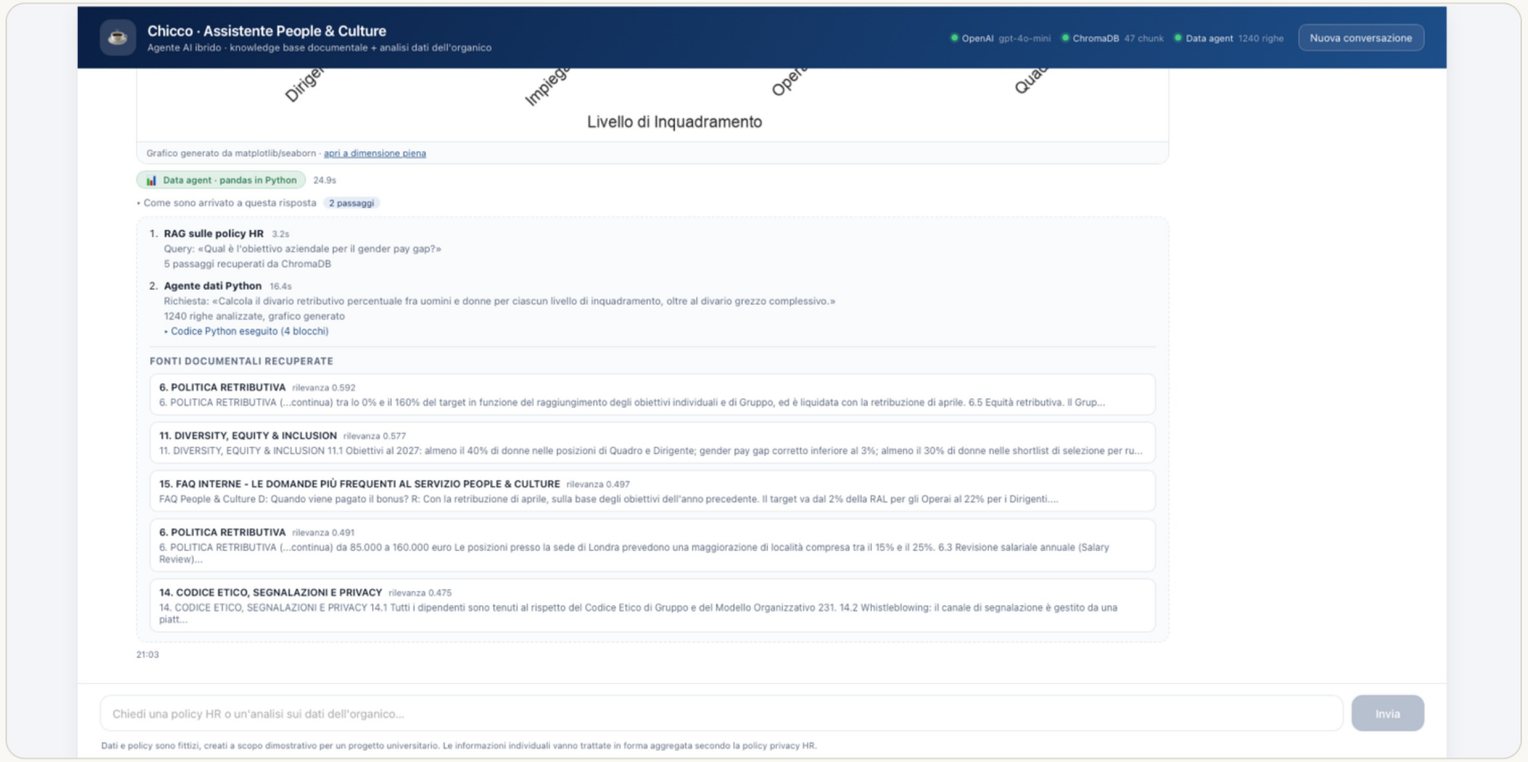
2 passaggi · 24,9 s

1. **RAG sulle policy HR**
   «qual è l'obiettivo aziendale per il gender pay gap?» — cinque passaggi recuperati da ChromaDB.

2. **Agente dati Python**
   «calcola il divario retributivo percentuale fra uomini e donne per ciascun livello di inquadramento, oltre al divario grezzo complessivo».

> La seconda richiesta **non è la domanda dell'utente**: l'orchestratore l'ha costruita sulla definizione appena letta nella policy, perché le due grandezze siano davvero confrontabili.

Ogni passaggio resta ispezionabile: la query, le fonti con il loro punteggio e il codice Python realmente eseguito.



DEMO · AFFIDABILITÀ

Quando la policy non copre il punto, l'agente lo dice

Knowledge base · RAG

La domanda

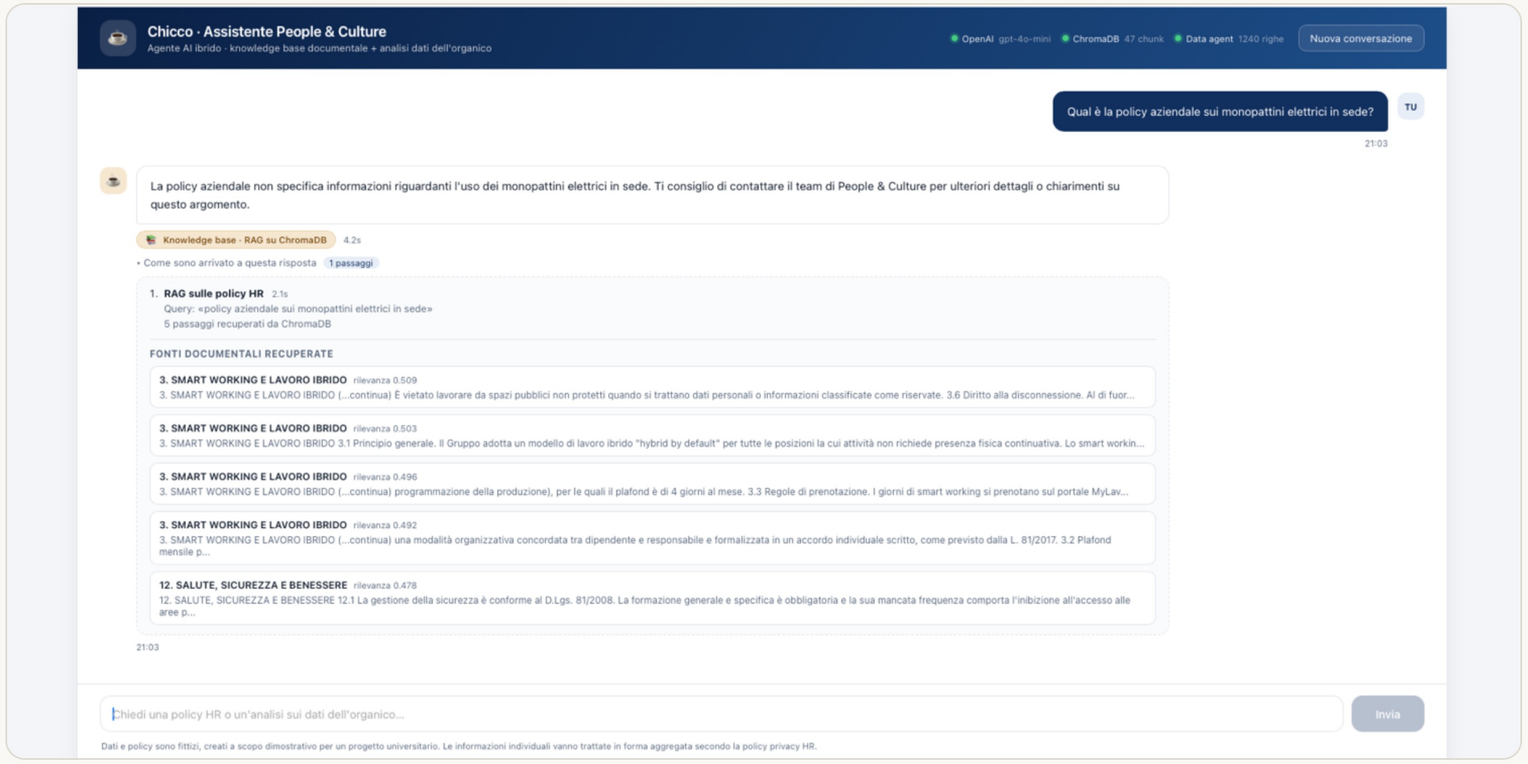
«Qual è la policy aziendale sui monopattini elettrici in sede?» — un tema plausibile, che il manuale non tratta.

Cosa succede

La ricerca semantica restituisce comunque cinque passaggi: sono i più vicini, ma nessuno risponde. È il caso in cui un assistente mal costruito inventa.

> Il prompt vieta l'uso di conoscenza generica sul diritto del lavoro: se la risposta non è nel contesto recuperato, l'agente deve dichiararlo e rimandare a People & Culture.

Per un responsabile HR è la garanzia che conta davvero: sapere quando non fidarsi è più utile di una risposta sempre pronta.



SFIDE TECNICHE · I

Tre errori che un numero plausibile nasconde

Emersi solo testando il sistema con le risposte alla mano e confrontandole con il calcolo diretto su pandas. Nessuno dei tre faceva fallire il programma.

IL SINTOMO

Turnover al 20% su un reparto e 0% su tutti gli altri

Causa. L'agente applicava *dropna* anche a *data_cessazione*, che è vuota per definizione su chi è ancora in forza: spariva l'81% della popolazione.
Soluzione. La differenza fra «dato mancante» e «vuoto che significa qualcosa» è ora esplicita nello schema, con un controllo di plausibilità obbligatorio prima della risposta.

IL SINTOMO

Un divario in euro confrontato con un obiettivo in percentuale

Causa. La policy fissa il 3% sul divario *corretto* a parità di ruolo e livello; l'agente misurava il divario grezzo. Stesso difetto sullo smart working, con una media unica contro plafond differenziati per funzione.
Soluzione. È l'orchestratore, che ha appena letto la policy, a formulare la richiesta di analisi nei termini della soglia: stessa unità, stesso livello di disaggregazione.

IL SINTOMO

Gender pay gap al 6,55% invece che al 6,15%

Causa. Il divario veniva calcolato mettendo al denominatore la retribuzione femminile, il che gonfia il risultato e lo rende non confrontabile con i benchmark.
Soluzione. Convenzione Eurostat fissata nel prompt: *(media uomini – media donne) / media uomini*.

SFIDE TECNICHE · II

Le istruzioni importanti non si chiedono, si impongono

Una regola scritta dentro un prompt lungo viene rispettata *a volte*. Due comportamenti sono stati spostati dal prompt al codice, dove non sono più negoziabili.

IL CASO PIÙ GRAVE

Una RAL media inventata: 36.000 € invece di 48.258 €

Il blocco di codice finiva con un'assegnazione, che in Python non restituisce nulla. L'agente non aveva mai *visto* i numeri e, dovendo scrivere la risposta, li ha riempiti con cifre plausibili prese dalle righe di esempio. Il grafico, calcolato sui dati veri, era corretto: l'errore stava solo nel testo.

PythonReplConEco
Il REPL non restituisce mai un'osservazione vuota: al suo posto arriva l'istruzione a rieseguire il blocco terminandolo con il valore da osservare.

IL CASO RICORRENTE

Grafici con una sola barra, a intermittenza

Alla domanda «e per il Marketing?» l'agente disegnava una barra isolata, che non comunica nulla. La regola nel prompt veniva seguita in alcune esecuzioni e ignorata in altre: imprevedibile, quindi inutile in una demo e pericolosa in produzione.

PyplotConGuardia
savefig rifiuta una figura con una sola categoria e solleva un errore che spiega come correggere: l'agente lo legge come output del proprio codice e ridisegna il confronto completo.

Entrambi i guardrail sono deterministici e lasciano all'agente la libertà di *come* correggersi: **non riducono l'autonomia, delimitano gli esiti inaccettabili.**

DOPO LA REVISIONE TECNICA

Sette segnalazioni, sette interventi misurati

Il progetto è passato da una revisione tecnica esterna. Nessun intervento è stato dato per buono: ciascuno ha una prova di esecuzione, non solo una riga di codice.

MANUTENZIONE E SCALA

I grafici non si accumulano più

Un task in background rimuove i PNG oltre le 24 ore, all'avvio e ogni ora.

✓ file invecchiato a mano → rimosso, 43 KB liberati

Cache delle risposte

A livello di turno, non di tool: l'orchestratore riformula la domanda, quindi una cache più in basso non farebbe mai centro. La chiave tiene conto della conversazione e della firma delle fonti.

✓ 13,9 s al primo giro, immediata ai successivi

Il dataset vive su DuckDB

Normalizzato una volta sola. Oltre la soglia di memoria l'agente riceve un campione e interroga il motore in SQL.

✓ soglia forzata: l'agente è passato da SELECT … GROUP BY

CORRETTEZZA E SICUREZZA

Codice generato, ma contenuto

Analisi statica prima dell'esecuzione e builtins ridotti: niente import fuori whitelist, niente **eval**, **open** o attributi interni.

✓ il modello ha tentato **import os** 5 volte: bloccato ogni volta

Il troncamento non rompe il JSON

Si accorciano i singoli campi prima di serializzare, non la stringa già pronta.

✓ payload da 40.000 caratteri: sempre valido

Timeout allineati

Il data agent dichiara il proprio budget, il backend lo confronta col proprio all'accensione.

✓ Node 260s > data agent 195s, margine 65s

Errori riconosciuti per classe

Non più per testo del messaggio, che cambia con la versione o la lingua.

Restano dichiarati anche i limiti noti: il sandbox non impone ancora tetti di memoria e CPU. Sapere dove il sistema non arriva fa parte della consegna.

SECONDA REVISIONE · LE SOLUZIONI AVEVANO UN FONDO

Due risposte giuste che risolvevano solo metà del problema

La revisione successiva non ha trovato errori nuovi: ha guardato *dentro* le soluzioni del giro precedente e ha mostrato dove si fermavano.

LA CACHE NON ERA CONDIVISA

Viveva nella memoria di un processo

Due repliche del backend non si sarebbero viste, e ogni riavvio azzerava tutto. Più insidiosa la memoria conversazionale: dietro un bilanciatore, «e per il Marketing?» poteva arrivare all'istanza che non ricorda la domanda precedente.

Cache e sessioni su Redis, dietro la stessa interfaccia: senza configurazione restano in memoria, con **REDIS_URL** sono condivise.

Alla prima prova, spegnendo Redis la richiesta è rimasta **appesa per sempre**: il client accodava i comandi. Ora ogni operazione ha un tetto di tempo oltre il quale si prosegue in memoria.

✓ backend riavviato da zero: risposta dalla cache in 0,1 s
✓ Redis spento: l'agente risponde comunque in 7,4 s

DUCKDB RISOLVEVA SOLO LA LETTURA

Il CSV passava ancora da pandas

Per costruire la tabella, il file veniva comunque materializzato per intero in un DataFrame e solo dopo scritto. Il picco di memoria restava proporzionale alla dimensione del file: esattamente il problema che lo store doveva togliere.

La pulizia è diventata una query. Trim, deduplica, categorie, importi con separatori misti, quattro formati di data, range e colonne derivate: tutto in SQL, con il file letto in streaming.

Riscrivere regole di dominio in un altro linguaggio è il momento in cui nascono differenze silenziose. La versione pandas resta come **oracolo di un test di parità** cella per cella.

✓ picco di allocazioni Python: da 1,2 MB a 0,0 MB
✓ test di parità: stesso dataset, cella per cella

Entrambe sono emerse **provando a rompere** ciò che funzionava — e la stessa verifica ha scoperto un difetto latente: una list comprehension non vedeva le variabili assegnate poco sopra, e l'agente ritentava fino a esaurire il budget, restituendo la risposta senza grafico. Una soluzione non verificata sotto stress è solo un'intenzione.

IN CHIUSURA

Cosa resta in mano a People & Culture

Un unico punto d'ingresso

Policy e dati nella stessa conversazione, senza decidere in anticipo quale dei due serve.

Risposte difendibili

Ogni numero è ricalcolato alla fonte e accompagnato dal codice; ogni regola dalla sua sezione di manuale.

Una base estendibile

Aggiungere un documento o un dataset non cambia l'architettura: cambia solo cosa l'agente può consultare.

SICUREZZA E GOVERNO DEL DATO

Nessuna credenziale nel codice: un unico file **.env** alla radice, escluso dal versionamento e consegnato a parte.
L'esecuzione di codice è confinata nel microservizio Python, che non è esposto all'esterno e riceve solo la domanda in linguaggio naturale.
I dati individuali restano trattati in forma aggregata, come prevede la policy privacy HR.

DOVE PUÒ ANDARE

Altri corpus documentali: contratti integrativi, verbali di accordo sindacale, manuali di sicurezza.
Altri dataset: assenze, recruiting, costi del personale.
Profili di accesso differenziati, così che un responsabile veda solo il perimetro del proprio team.
Valutazione automatica delle risposte su un set di domande di controllo.

CODICE, DATI E DOCUMENTAZIONE

github.com/GolDRoger7/belmoro-hr-agent

Istruzioni complete di installazione e avvio nel README.
Riproducibilità verificata clonando il repository da zero.